

DEVELOPER HANDOFF | ENGLISH + العربية

NAFS Prep Hostinger Deployment Guide

Step-by-step setup for the application, PostgreSQL database, uploaded images, and custom domain.

Next.js 16

Node.js

Prisma 7

PostgreSQL

Hostinger

Recommended setup: Hostinger Node.js Web App hosting + an external PostgreSQL database (for example, Supabase) + durable cloud image/file storage. Hostinger Web and Cloud plans do not provide PostgreSQL databases; PostgreSQL on Hostinger requires a VPS.

Prepared 28 September 2026 from the project files and current Hostinger Help Center guidance. Hostinger dashboard labels can change; follow the current prompts in hPanel.

1. Choose the hosting and database

2. Prepare the source code

3. Configure production secrets

4. Deploy the app

5. Attach the domain and HTTPS

6. Images, uploads, and checks

1. Choose the hosting and database

1. For the simplest deployment, use Hostinger **Node.js Web App** hosting on an eligible Business or Cloud plan. Hostinger lists Next.js as supported. Choose Node.js **22.x** (or another supported LTS version tested by the developer).
2. Keep PostgreSQL on Supabase or another managed PostgreSQL provider. This project uses PostgreSQL through Prisma and the pg driver. Hostinger Web/Cloud database service is MariaDB/MySQL; it is not a drop-in replacement. If PostgreSQL must run on Hostinger, choose a VPS and have the developer manage PostgreSQL, firewall, backups, updates, and monitoring.
3. Ask the developer to create a production database or a production database branch separate from development. Confirm its region, network access, backup plan, and connection string.

Hostinger plan distinction: managed Node.js Web App hosting is easier to operate. A Hostinger VPS offers more control, including PostgreSQL, but the server needs ongoing administration.

2. Prepare the source code

1. Give the developer the complete project repository, including `package.json`, `package-lock.json`, `src/`, `prisma/schema.prisma`, and every folder under `prisma/migrations/`. Use a private GitHub repository and grant access to the developer and Hostinger.
2. Do **not** send or commit the local `.env` file. Share secret values through a secure password manager or enter them directly in Hostinger.
3. Tell the developer the app's quiz CSV refers to files under `/image-source/`. The source folder contains about 1,285 files and is roughly 2.5 GB; it is not currently tracked in Git. Do not assume a normal Git deployment will include those files. Prefer moving the referenced images to durable object/image storage, then update each question's database URL to its public HTTPS URL. If keeping local files instead, the deployment package must explicitly include the full folder and Hostinger storage limits must be checked first.
4. Use persistent external storage for teacher-uploaded study guides and generated images too. The current routes write these to local `public/` directories, which can be tied to a deployment's local filesystem and should not be treated as durable storage across deployments.
5. Include the image URL normalization fix in the handoff. CSV values such as `image-source/example.png` need a leading slash when served locally; the app now normalizes that path for new imports and quiz display.
6. Before production, the developer should review authentication, role permissions, file-upload validation, and whether production seed data is needed. Do not run a seed script blindly against an existing live database.

3. Configure production secrets

Add these as Hostinger environment variables (or in the VPS process manager). Use production values and never put them in the repository.

Variable	Production value / purpose
DATABASE_URL	Production PostgreSQL URL. Use the provider's recommended connection/pooler string. URL-encode special characters in the password.
NEXTAUTH_URL	<code>https://your-domain.example</code> (the final canonical HTTPS domain).
NEXTAUTH_SECRET	A long, random secret generated for production. Do not reuse a public or development value.
GEMINI_API_KEY	Required for AI quiz and image-generation features.
GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET	Configure valid Google OAuth credentials; the current auth configuration registers Google as a provider.

For Google OAuth, add this authorized redirect URI in Google Cloud Console: `https://your-domain.example/api/auth/callback/google`. If the app uses additional third-party AI or storage services after the developer's review, add only the corresponding required keys.

4. Deploy the app

1. In Hostinger hPanel, open **Websites** → **Add Website** → **Node.js Web App / Deploy Web App**, then connect the private GitHub repository (or upload a ZIP if GitHub deployment is unavailable).
2. Select the production branch. Set framework to Next.js if not detected, Node.js runtime to 22.x, and add the environment variables before building.
3. Use the project scripts: build command `npm run build`; start command `npm start`. The build script runs `prisma generate` before `next build`.
4. Apply database migrations **before opening the production app**. With a database backup available, the developer should run `npx prisma migrate deploy` from a trusted machine/runner that has the production `DATABASE_URL` and the same repository checkout. For Hostinger VPS, this can be run from the server after code and secrets are configured. Review pending migrations and back up first; do not use `prisma migrate reset` on production.
5. Start the deployment and review Hostinger build logs. Confirm Next.js reaches a successful production build and the app process is running.

```
npm ci
npx prisma migrate deploy
npm run build
npm start
```

On managed Hostinger Node.js hosting, deployment runs installation/build steps in hPanel. Use the dashboard's supported build configuration; production migrations may be run from a separate secure runner if SSH is unavailable.

5. Attach the domain and HTTPS

1. In the Node.js website dashboard, choose **Connect domain** and enter the domain/subdomain, such as `nafs.example.com`.
2. Follow Hostinger's DNS instructions. If DNS is managed elsewhere, add the exact A/CNAME records Hostinger displays at that DNS provider. Do not delete existing mail-related MX/TXT records.
3. Wait for DNS propagation, which may take up to 24 hours. Confirm the website opens on the custom domain.
4. Hostinger says it installs SSL automatically after domain connection completes. Verify `https://` loads without a certificate warning, and verify `NEXTAUTH_URL` and the Google callback use that same HTTPS domain.

6. Images, uploads, and final checks

- Open one quiz that has an image; verify its image URL returns an image successfully. If using cloud storage, verify the database rows contain the full public HTTPS URLs.
- Test student, teacher, and admin sign-in; create/open a quiz; import a small test CSV; submit a quiz; and verify dashboard/API access.
- Test study-guide upload and image generation. Confirm newly uploaded/generated files remain available after a redeployment; if not, storage is still local and must be moved to persistent storage.
- Check Hostinger runtime logs and the browser console for 404 image errors, auth callback errors, Prisma errors, and missing environment variables.
- Set up database backups and a rollback/redeploy plan. Keep the old production site/database available until the new deployment has been accepted.

Official Hostinger references

1. [Deploy a Node.js website in Hostinger](#) - supported plans, GitHub/ZIP setup, build settings, and runtime.
2. [Connect Supabase PostgreSQL to a Hostinger Node.js app](#).
3. [Hostinger database support](#) - PostgreSQL requires VPS hosting; Web/Cloud plans support MariaDB/MySQL.
4. [Connect a custom domain and SSL to a Node.js app](#).

نشر تطبيق NAFS Prep على Hostinger

خطوات تسليم المطور لإعداد التطبيق وقاعدة PostgreSQL والصور والنطاق.

الإعداد المقترح: استضافة تطبيقات Node.js المُدارة من Hostinger، مع PostgreSQL خارجي مثل Supabase، وتخزين دائم للصور والملفات. خطط Hostinger Web و Cloud لا توفر PostgreSQL؛ تشغيل PostgreSQL على Hostinger يتطلب VPS.

١. اختيار الاستضافة وقاعدة البيانات

1. للنشر الأسهل، اختر استضافة Node.js Web App في خطة Business أو Cloud مؤهلة. Hostinger تذكر Next.js ضمن الأطر المدعومة. اختر Node.js إصدار x.22 أو إصدار LTS مدعوماً اختيره المطور.
2. استمر باستخدام PostgreSQL على Supabase أو مزود PostgreSQL مُدار آخر. المشروع يستخدم PostgreSQL مع Prisma ومكتبة pg. قاعدة بيانات Hostinger في خطط Web و Cloud هي MariaDB/MySQL وليست بديلاً مباشراً. إذا أردت PostgreSQL على Hostinger، اختر VPS وتأكد أن المطور سيتولى إدارة قاعدة البيانات والجدار الناري والتحديثات والنسخ الاحتياطية.
3. اطلب من المطور إعداد قاعدة إنتاج مستقلة عن التطوير، أو فرع إنتاج مستقل، والتحقق من المنطقة الجغرافية والاتصال والنسخ الاحتياطية ورابط الاتصال.

٢. تجهيز ملفات المشروع

1. زود المطور بمستودع المشروع كاملاً، بما فيه package.json و package-lock.json ومجلد /src وملف prisma/schema.prisma وجميع مجلدات ./prisma/migrations. استخدم مستودع GitHub خاصاً وامنح المطور و Hostinger صلاحية الوصول.
2. لا ترسل ملف .env المحلي ولا ترفعه إلى GitHub. شارك الأسرار عبر مدير كلمات مرور آمن أو أدخلها مباشرة في Hostinger.
3. أخبر المطور أن ملف CSV يشير إلى صور داخل ./image-source/. يحتوي مجلد المصدر على نحو ١,٢٨٥ ملفاً وحجمه يقارب ٢,٥ غيغابايت، وهو غير محفوظ حالياً في Git. لذلك لن تنتقل الصور تلقائياً مع النشر من Git. الأفضل رفع الصور إلى تخزين سحابي دائم، ثم تحديث روابط الصور في قاعدة البيانات إلى روابط HTTPS عامة. وإذا تقرر إبقاؤها كمفات محلية، يجب تضمين المجلد كاملاً في حزمة النشر والتحقق أولاً من حدود مساحة Hostinger.
4. استخدم تخزيناً خارجياً دائماً لملفات الأدلة الدراسية والصور المنشأة أيضاً. المسارات الحالية تحفظ هذه الملفات داخل مجلدات /public محلياً، ولا ينبغي اعتبارها دائمة بعد كل عملية نشر.
5. تأكد من تضمين إصلاح روابط الصور. قيم CSV مثل image-source/example.png تحتاج إلى شرطة مائلة أولى عند عرضها محلياً؛ التطبيق يضيفها الآن عند الاستيراد وعرض الاختبارات.
6. قبل الإنتاج، على المطور مراجعة تسجيل الدخول والصلاحيات والتحقق من الملفات المرفوعة وبيانات البذور. لا تشغل سكربت seed عشوائياً على قاعدة الإنتاج الحالية.

٣. إعداد أسرار الإنتاج

أضف القيم التالية إلى متغيرات البيئة في Hostinger أو مدير عمليات VPS. استخدم قيماً خاصة بالإنتاج ولا تضعها في المستودع.

الغرض	المتغير
رابط PostgreSQL للإنتاج، باستخدام صيغة الاتصال أو pooler التي يوصي بها مزود قاعدة البيانات. شقّر الرموز الخاصة في كلمة المرور داخل الرابط.	DATABASE_URL
https://your-domain.example أي النطاق النهائي الآمن.	NEXTAUTH_URL
سر طويل وعشوائي للإنتاج. لا تستخدم قيمة التطوير ولا تنشره.	NEXTAUTH_SECRET
مطلوب لميزات إنشاء الاختبارات والصور بالذكاء الاصطناعي.	GEMINI_API_KEY
بيانات Google OAuth؛ إعداد المصادقة الحالي يضيف Google كمزود.	GOOGLE_CLIENT_ID وGOOGLE_CLIENT_SECRET

في Google Cloud Console أضف رابط إعادة التوجيه التالي: `https://your-domain.example/api/auth/callback/google`. إذا أكد المطور استخدام خدمات ذكاء اصطناعي أو تخزين إضافية، أضف مفاتيحها المطلوبة فقط.

٤. نشر التطبيق

1. من لوحة Hostinger افتح **Node.js Web App / Deploy Web App** ← **Add Website** ← **Websites**، ثم اربط مستودع GitHub الخاص أو ارفع ملف ZIP إذا تعذر الربط.
2. اختر فرع الإنتاج. اختر Next.js إذا لم يتعرف عليه النظام، وحدد Node.js 22.x، وأضف متغيرات البيئة قبل عملية البناء.
3. استخدم أوامر المشروع: أمر البناء `npm run build` وأمر التشغيل `npm start`. أمر البناء يشغّل `prisma generate` ثم `next build`.
4. طبقّ ترحيلات قاعدة البيانات قبل فتح التطبيق للإنتاج. بعد أخذ نسخة احتياطية ومراجعة التغييرات، يشغّل المطور `npx prisma migrate deploy` من جهاز أو بيئة موثوقة لديها رابط الإنتاج ونسخة المستودع نفسها. على VPS يمكن تشغيله من الخادم بعد إعداد الكود والأسرار. لا تستخدم `prisma migrate reset` على الإنتاج.
5. ابدأ النشر وراجع سجلات البناء في Hostinger. تأكد من نجاح بناء Next.js ومن عمل التطبيق.

```
npm ci
npx prisma migrate deploy
npm run build
npm start
```

في استضافة Node.js المُدارة، تتم خطوات تثبيت الحزم والبناء من لوحة Hostinger. إذا لم يتوفر SSH، شغّل ترحيلات قاعدة الإنتاج من بيئة آمنة منفصلة قبل النشر.

٥. ربط النطاق وتفعيل HTTPS

1. في لوحة تطبيق Node.js اختر **Connect domain** وأدخل النطاق أو النطاق الفرعي، مثل `nafs.example.com`.
2. اتبع تعليمات DNS التي تعرضها Hostinger. إذا كانت إدارة DNS لدى شركة أخرى، أضف سجلات A أو CNAME المطابقة في لوحة تلك الشركة. لا تحذف سجلات البريد MX أو TXT الموجودة.
3. انتظر انتشار DNS؛ قد يستغرق ذلك حتى ٢٤ ساعة. بعدها افتح الموقع عبر النطاق الجديد.
4. تذكر Hostinger أنها تثبت SSL تلقائياً بعد اكتمال ربط النطاق. تحقق من عمل `https://` دون تحذير، ومن تطابق `NEXTAUTH_URL` و رابط Google مع النطاق الآمن نفسه.

٦. الصور والملفات والفحص النهائي

- افتح اختباراً يحتوي على صورة وتأكد من فتح رابطها. إذا استخدمت تخزيناً سحابياً، تحقق من أن قاعدة البيانات تحتوي روابط HTTPS عامة كاملة.
- اختبر تسجيل دخول الطالب والمعلم والمدير، وفتح اختبار، واستيراد CSV تجريبي صغير، وتسليم اختبار، والوصول إلى لوحة التحكم وواجهات API.
- اختبر رفع دليل دراسي وإنشاء صورة. تأكد أن الملفات تبقى متاحة بعد إعادة النشر. إذا اختفت، فهذا يعني أن التخزين ما زال محلياً ويجب نقله إلى تخزين دائم.
- راجع سجلات Hostinger وأدوات المطور في المتصفح بحثاً عن صور 404 أو أخطاء تسجيل الدخول أو Prisma أو متغيرات البيئة الناقصة.
- فعل النسخ الاحتياطية لقاعدة البيانات وجّهز خطة رجوع. احتفظ بالموقع وقاعدة البيانات القديمة حتى قبول النشر الجديد.

مراجع Hostinger الرسمية

1. [Hostinger على Node.js نشر موقع](#) - الخطط المدعومة والربط بـ GitHub وإعدادات البناء.
2. [Hostinger على Node.js بتطبيق Supabase PostgreSQL ربط](#).
3. [Hostinger - PostgreSQL قواعد البيانات المدعومة في VPS](#)؛ وخطط Web و Cloud تدعم MariaDB/MySQL.
4. [Node.js بتطبيق SSL ربط نطاق مخصص و](#).